Diseño Físico de BBDD I

UD4.1

Introducción

Introducción I

Los objetivos principales de la etapa de diseño físico de una base de datos son:

- Traducir el modelo lógico a las características del SGBD elegido.
- Diseñar las tablas bases y sus restricciones.
- Optimizar la organización de la información de la BD.
- Diseñar mecanismos de seguridad, control y monitorización del sistema.

Introducción II

Todo lo anterior se analizará por fases.

1. Traducción del modelo lógico al modelo físico (DDL).
2. Desarrollo y manipulación de datos en la BD (DML).
3. Control y gestión del sistema (DCL).
4. Lenguaje de control de transacciones (TCL).

Fase 1. Traducción del modelo lógico al modelo físico

Lenguaje de Definición de Datos (DDL)

Lenguaje de Definición de Datos (DDL) por

Daenerys Targaryen

De la casa Targaryen, khalesi de los dothraki, reina de los ándalos, rhoynar y los primeros hombres. Señora y protectora del reino.

Hija de la Tormenta, La que no Arde, Rompedora de Cadenas y Madre de Dragones.

CREALDRO

CREATE (**CREA**LDRO)

ALTER (CRE**AL**DROP)

DROP (CREAL**DRO**)

¡Ojo con dejar suelto al dragón!

¿Por dónde empezamos?

¿Qué modelado hemos elegido?

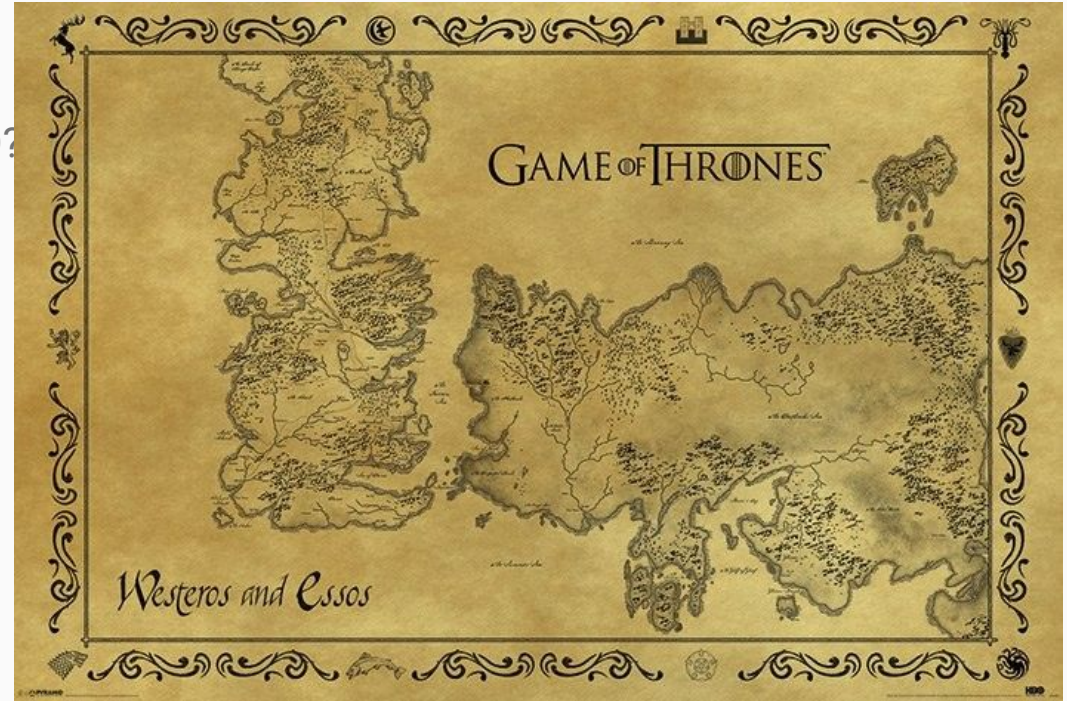
¿Cuál será el tamaño de nuestra BBDD?

¿Será centralizada o distribuida?

¿Cuál será el volumen de accesos concurrentes?

¿Se guardarán datos de geolocalización? ¿Cada cuánto?

...



Una vez escogido el SGBD

Ahora se aplicarán unas últimas transformaciones específicas al SGBD escogido para obtener un esquema final de la BBDD desde su modelo lógico (definiciones últimas de tablas, restricciones, optimización de la información, etc.).

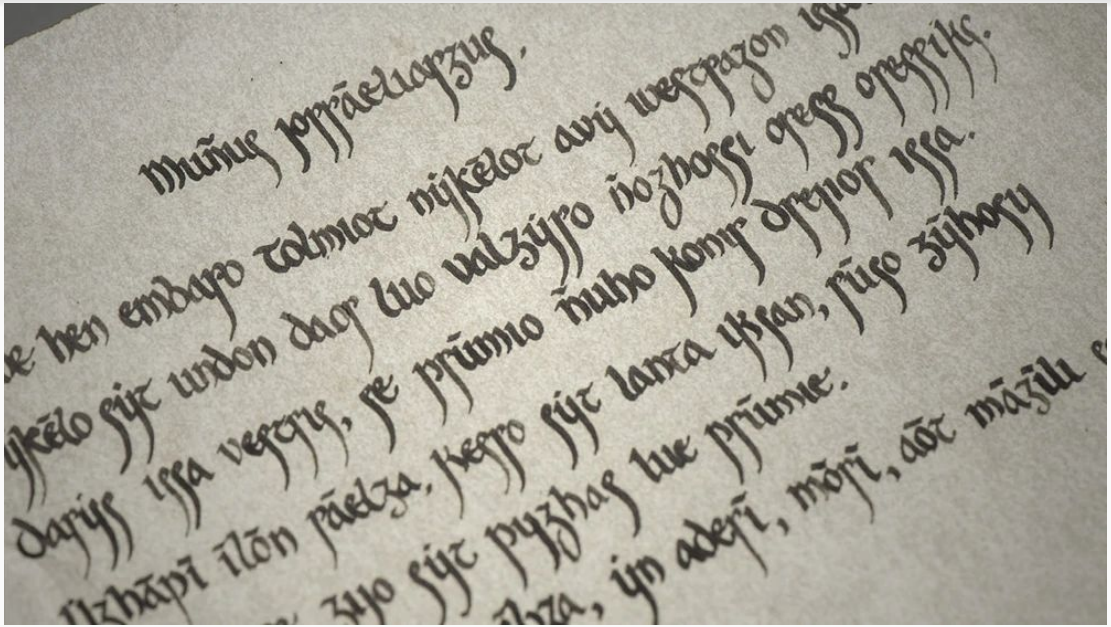
Lenguaje de las BBDD relacionales

Lenguaje de las BBDD relacionales I

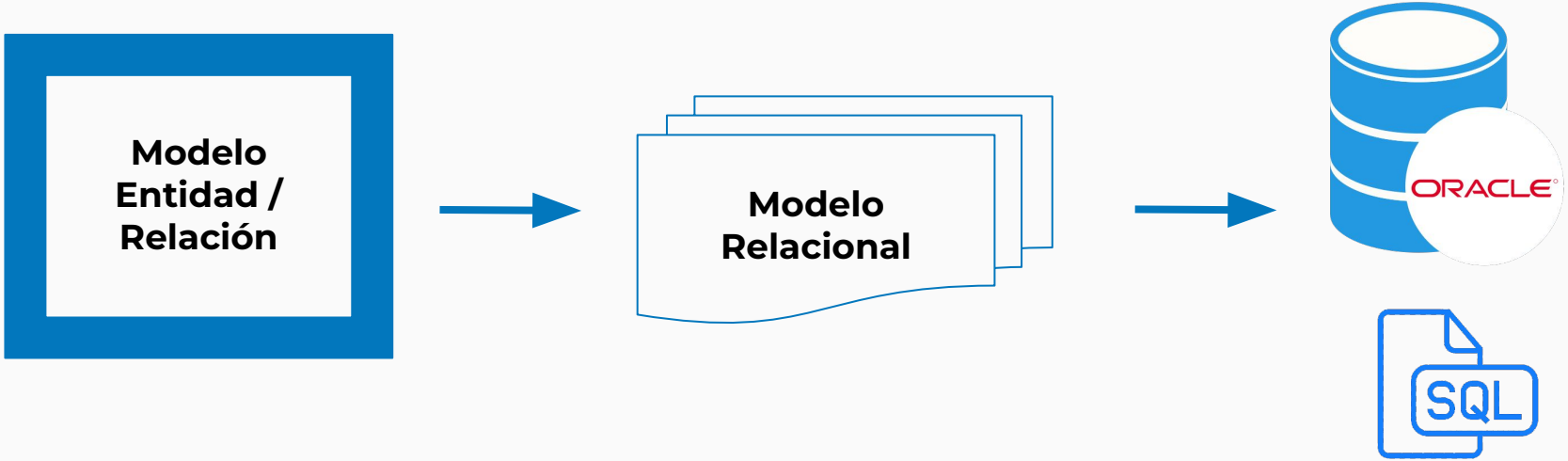
Para gestionar cualquier BBDD relacional el lenguaje más extendido y usado es

SQL.

Gracias a este lenguaje podemos construir y gestionar una Base de Datos y **la información** contenida.



Lenguaje de las BBDD relacionales II



Algunas claves básicas de SQL I

Hasta que que el **parser** no encuentra “**;**” no se ejecuta el **comando SQL**:

Ejemplo 1.

```
SELECT *
FROM Profesores
WHERE Nombre='Javier';
```

Ejemplo 2.

```
SELECT * FROM Profesores
WHERE Nombre='Javier';
```

Ejemplo 3.

```
SELECT * FROM Profesores
WHERE Nombre='Javier'
;
```

Los comandos pueden escribirse tanto en **MAYUS** cómo en **minúsculas**, pero cuidado con los datos, ahí si existen diferencias:

```
SELECT * FROM Profesores WHERE Nombre='Javier';
select * from Profesores where Nombre='Javier';
SELECT * from Profesores WheRE Nombre='JAVier';
```

Algunas claves básicas de SQL II

Los comandos SQL de definición de elementos en una BBDD **no pueden ser anulados** (ROLLBACK).

P.e. Una vez creada (CREATE) una tabla en la BD, sólo podrá ser eliminada mediante el comando (DROP).

Estos elementos pueden ser: tablas, columnas, restricciones, sinónimos, índices, vistas, vistas actualizables, vistas materializadas, disparadores, espacios de tablas, segmentos, extensiones, procedimientos, funciones, cursores, usuarios, roles, privilegios, perfiles y un largo etcétera.

Algunas claves básicas de SQL III

Existen unas reglas básicas para poder nombrar elementos en una BD:

- Deben comenzar por una letra A…Z ó a…Z nunca por un número.
- Max. 30 caracteres y sin símbolos especiales.
- Ningún elemento (u objeto) se puede llamar igual que otro del mismo tipo.
- Si el nombre tiene espacios, se debe escribir entre comillas simples. P.e. ‘Profesor Asignatura’
- Se debería evitar llamar dos elementos iguales pero uno en mayúsculas y otro en minúsculas.

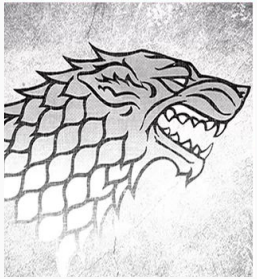
Los tipos de datos

Tipos de datos

Al crear una nueva columna se debe indicar el dominio o tipo de datos que la contiene.

Esto se puede realizar de forma **explícita** (con unos valores a elegir) o de forma **implícita**, indicando el **tipo de datos** que podrán ser admitidos.

Tipos de datos numéricos



- NUMBER(E,D) - Puede contener tanto valores enteros como decimales (Sueldo, Hijos, etc.).
- BINARY_FLOAT - Puede contener val. numéricos con coma flotante hasta 32 bits (Precio, Beneficios, etc.).
- BINARY_DOUBLE - El doble que BINARY_FLOAT, es decir, hasta 64 Bits (Capital, ‘Precio Bitcoin’, etc.).

Tipos de datos textos

- VARCHAR2(N) - Puede contener una cadena de hasta N caracteres (Nombre, Apellidos, DNI, etc.).
- CHAR(N) - Contiene siempre N caracteres alfanuméricos (Código, Clave, etc.).
- LONG - Admite cadenas de hasta 2 GB (Foto, Video, etc.).

Tipos de datos fecha y hora

- DATE - Almacena la fecha y hora (‘Fecha de Nacimiento’, Fecha, etc..).
- TIMESTAMP[(N)] - Almacena fecha y hora con N dígitos (‘Fecha de actualización’, etc.).
- INTERVAL YEAR(N) TO MONTH (M) | DAY (D) TO SECOND(S) - Puede ser año y mes o día y segundo (Duración, Momento, etc.).

Otros tipos de datos

- BLOB, CLOB, BFILE - Datos de gran tamaño. (Archivos, Imagen, etc..).

Expresiones relacionales y lógicas

Expresiones relacionales y lógicas I

Para **comparar valores** y evaluar si son iguales o diferentes; mayores o menores se usan las **expresiones relacionales.**

A continuación se analizan en una tabla los símbolos que representan estas operaciones relacionales.

Expresiones lógicas y relacionales II (**Operadores relacionales**)

Operador relacional	Representación	Ejemplo
Igual a	=	Edad = 18
Distinto de	!=, <>	Edad != 18
Mayor que	>	Edad > 18
Menor que	<	Edad < 18
Mayor igual que	>=	Edad >= 18
Menor igual que	<=	Edad <= 18

Expresiones relacionales y lógicas III

Las expresiones lógicas son aquellas que tras su evaluación, devuelven un cierto valor (**true o false**).

También puede contener operaciones lógicas con sus operadores básicos:

Not, Or y And

Expresiones lógicas y relacionales IV (**Operadores lógicos I**)

Operación	Resultado	Descripción	Ejemplo
NOT true	False	No verdadero es falso.	NOT(edad>100) será cierto mientras edad sea menor o igual que 100.
NOT false	True	No falso es verdadero.	NOT(edad<100) Cierto para edad mayor o igual de 100.
false OR false	False	Si algo es falso o algo es falso, todo es falso.	a>5 or b<10 es falso para a=4 y b=12
false OR true	True	Si algo es falso o algo es verdadero, todo es verdadero.	a>5 or b<10 es cierto para a=4 y b=8
true OR false	True	Si algo es verdadero o algo es falso, todo es verdadero.	a>5 or b<10 es cierto para a=6 y b=12
true OR true	True	Si algo es verdadero o algo es verdadero, todo es verdadero.	a>5 or b<10 es cierto para a=6 y b=8

Expresiones lógicas y relacionales V (**Operadores lógicos II**)

Operación	Resultado	Descripción	Ejemplo
false AND false	False	Si algo es falso y algo es falso, todo es falso.	a>5 and b<10 es falso para a=4 y b=12
false AND true	False	Si algo es falso y algo es verdadero, todo es falso.	a>5 and b<10 es falso para a=4 y b=8
true AND false	False	Si algo es verdadero y algo es falso, todo es falso.	a>5 and b<10 es falso para a=6 y b=12
true AND true	True	Si algo es verdadero falsoalgo es verdadero, todo es verdadero.	a>5 and b<10 es cierto para a=6 y b=8
Multiples expresiones		Se debe tener en cuenta el orden de las expresiones. Se deberá usar paréntesis al igual que en la aritmética.	n1=1 OR n1=20 AND n2=20 (n1=1 OR n1=20) AND n2=20

Expresiones lógicas y relacionales VI (**Ejemplos**)

Acción	Comando
Mostrar los regitros de la tabla Asignatura cuyo código sea mayor que 5.	SELECT * FROM Asignatura WHERE CodAsig > 5
Mostrar todos los regitros de la tabla Asignatura cuyo código sea mayor que 2 y menor que 5.	SELECT * FROM Asignatura WHERE CodAsig > 2 AND CodAsig < 5
Mostrar el campo Nombre con código igual a 1 en la tabla Asignatura.	SELECT Nombre FROM Asignatura WHERE CodAsig = 1
Mostrar los registros de Asignatura que el número de horas sea mayor que 60 y menor que 200.	SELECT * FROM Asignatura WHERE NumHoras > 60 AND NumHoras < 200
Mostrar los registros de Asignatura que el código del ciclo sea el 1 y el número de horas sea mayor o igual que 100 o menor o igual que 200.	SELECT * FROM Asignatura WHERE CodCiclo = 1 AND NumHoras >= 100 OR NumHoras <= 200

SGBD ORACLE

¿Qué es Oracle? ¿Qué ventajas tiene? I

Oracle es un Sistema de Gestión de Bases de Datos Relacionales (SGBD)

- Soportado por la mayoría de las plataformas de hardware y software.
- Su escalabilidad.
- Desarrollo en el servidor y en el cliente.
- Herramientas propias de programación y el diseño.
- Transacciones sobre la base de lecturas y escrituras diferidas y el sistema se configura:
    - multi-hilo con servidores compartidos (reduce los requerimientos de RAM),
    - servidor dedicado (garantiza la atención inmediata de una solicitud)
    - un híbrido de las anteriores

¿Qué es Oracle? ¿Qué ventajas tiene? II

- Acceso concurrente de usuarios. Integridad, consistencia, seguridad y buenos tiempos de respuesta para el trabajo interactivo.
- Control de acceso (conjunto de privilegios, roles, y perfiles).
- Disparadores de BD y procedimientos empaquetados.
- ORACLE implementa controles de seguridad e integridad
- Replicación
- Implementa fuertes mecanismos de resguardo tanto para fallos de instancia (terminación anormal de sesiones de trabajo, procesos, etc.) como para fallos de disco.
- Se configura a través de un fichero de parámetros (init.ora) dinámico.

Nomenclatura ORACLE

- Elementos y objetos en el SGBD Oracle
- Un DICCIONARIO DE DATOS es un área del sistema dentro del entorno de la BD que contiene información acerca de los componentes de la base de datos.
  - El diccionario de datos es una de las herramientas más potentes de una base de datos.
  - Tanto los usuarios finales, los ingenieros de sistema, como los administradores, todos usan el diccionario de datos, aunque a veces no sean conscientes de ello.
  - El diccionario de datos está formado por un conjunto de tablas y vistas cuyos propietarios son SYS y SYSTEM.
- Otros elementos de un ORACLE son: TABLE, USER, ROL, PROFILE, DATABASE, TABLESPACE, TYPE, PROCEDURE, FUNCTION, INDEX, SYNONY, etc.

Entorno de trabajo y ejecución de scripts

Ejecución de scripts

Los comandos se pueden ejecutar de uno en uno o varios de forma secuencial contenidos en un archivo con **extensión .sql o scripts**.

Se deberá conectar con un usuario correctamente autenticado y con permisos antes de ejecutar el archivo en el sistema.

Entornos de ejecución

Las ventajas de usar SQL Developer son:

- No consume muchos recursos máquina.
- Ejecuta los comandos de una forma optimizada.
- Compatibles con los comandos SQL como los programas PL/SQL
- El cliente dispone de motores PL/SQL sin necesidad de consumir recursos del servidor

Creación de una BD

En MariaDB es una tarea muy simple, basta con ejecutar el comando con un usuario con provilegios:

CREATE DATABASE nombreBD

y ya estará creada en el sistema.

En el SGBD Oracle no es tan sencillo.

Las principales tareas de la etapa física serían:

- **Creación de la BD e instancias**
- **Conexiones / apertura / cierre**
- **Creación de tablespaces y archivos**
- **Creación de usuarios y perfiles**
- **Gestión de roles y privilegios**
- **Elección de tipos de datos en columnas**
- Creación de tablas y restricciones
- Codificación de campos candidatos
- Creación de estructuras de control de datos y procedimientos almacenados
- Creación y configuración de otras estructuras fundamentales de admon.

Creación de tablas y restricciones

Los lenguajes dentro de las BBDD

Lenguaje de definición de datos o Data Definition Language (DDL): Está compuesto por un conjunto de comandos que permite crear modificar y eliminar las estructuras de almacenamiento y los procedimientos para esta gestión y consultas SELECT. Principalmente son CREATE, ALTER Y DROP y son los denominados CREALDRO.

Lenguaje de manipulación de datos o Data Manipulation Language (DML): Está compuesto por comandos que permiten la gestión de los datos almacenados en la estructuras definidas por los comandos DDL. Los conmandos más importantes de este lenguaje son INSERT, UPDATE y DELETE y son los llamados INUPDEL.

Lenguaje de control de datos o Data Control Language (DCL): Compuesto por los comandos que permiten la gestión del acceso a la BD. Principalmente son REVOKE, GRANT, CREATE USER, CREATE PROFILE.

Lengaje de control de transacciones o Transaction Control Languages (TCL): Compustos por comandos que permiten gestionar las transicciones, es decir, los grupos de operaciones vistas como acciones atómicas en la BD. Los comandos principales son COMMIT, ROLLBACK y SAVEPOINT.

CREALDRO

CREATE

Permite la creación de cualquier elemento u objeto que se permita crear en el sistema.

ALTER

Permite la modificación de cualquier elemento u objeto que se permita cambiar en el sistema.

DROP

Permite la eliminación de cualquier elemento u objeto que se permita eliminar en el sistema.

Lenguaje de definición de datos (DDL)

CREATE (**CREA**LDRO)

Estos comandos se usan para crear cualquier objeto u elemento en la BBDD, p.e. CREATE TABLE. Algunos de los comandos CREATE más usados son:

CREATE TABLE, CREATE USER, CREATE VIEW, CREATE TABLESPACE, CREATE DATABASE, CREATE INDEX, CREATE SYNONYM, CREATE TYPE, CREATE FUNCTION, CREATE PROCEDURE, CREATE ROLE, CREATE PROFILE, etc.

ALTER (CRE**AL**DRO)

Estos comandos permiten modificar un objeto ya creado.

ALTER TABLE, ALTER USER, ALTER ROL, ALTER PROFILE, ALTER DATABASE, ALTER TABLESPACE, ALTER TYPE, ALTER PROCEDURE, ALTER FUNCTION, ALTER INDEX, ALTER SYNONY, etc.

DROP (CREAL**DRO**)

Para poder eliminar cualquier objeto u elemento de la BD se usará DROP y seguido el elemento a eliminar.

P.e. **DROP TABLE Asignatura** eliminará completamente esta tabla del sistema.

Profundizando en la creación de objetos de una Base de datos

Comando CREATE TABLE I

Este comando permite la creación de tablas en la base de datos, indicando todas su propiedades (ya sean atributos o restricciones).

```
CREATE TABLE Profesor(
        CodProf number(14),
        DNI varchar2(9),
        Nombre varchar2(30),
        Apellido1 varchar2(30),
        Apellido2 varchar2(30),
        Teléfono number(9),
        CONSTRAINT pk_Profesor PRIMARY KEY(CodProf)
);
```

Una vez creada la tabla en el esquema, se usa el comando **DESCRIBE Profesor;** para identificar la correcta creación de la misma.

Comando CREATE TABLE II

Del mismo modo, se podría crear la tabla Estudiante

```
CREATE TABLE Estudiante(
        CodEst number(14),
        DNI varchar2(9),
        Nombre varchar2(30),
        Apellido1 varchar2(30),
        Apellido2 varchar2(30),
        Teléfono number(9),
        CONSTRAINT pk_Estudiante PRIMARY KEY(CodEst)
);
```

Las columnas pueden llamarse igual al encontrarse en diferentes tablas.

Declaración de las restricciones

Del mismo modo, se podrían definir las restricciones dentro de las definiciones de columnas:

```
CREATE TABLE Estudiante(
        CodEst number(14) CONSTRAINT pk_Estudiante PRIMARY KEY,
        DNI varchar2(9),
        Nombre varchar2(30),
        Apellido1 varchar2(30),
        Apellido2 varchar2(30),
        Telefono number(9)
        );
```

O incluso ahorrarse las palabras ***CONSTRAINT pk_Estudiante***:

```
CREATE TABLE Estudiante(
        CodEst number(14) PRIMARY KEY,
        DNI varchar2(9),
        Nombre varchar2(30),
        Apellido1 varchar2(30),
        Apellido2 varchar2(30),
        Telefono number(9)
        );
```

Restricciones NOT NULL, UNIQUE y PRIMARY KEY

```
CREATE TABLE Profesor(
        CodProf number(14) PRIMARY KEY,
        Nombre varchar2(30) NOT NULL,
        Apellido1 varchar2(30) NOT NULL,
        Apellido2 varchar2(30),
        DNI varchar2(9) NOT NULL UNIQUE,
       fecha_nacimiento DATE,
        Teléfono number(9),
        CONSTRAINT pk_profesor PRIMARY KEY (CodProf)
);

CREATE TABLE Ejemplar(
        CodLibro number(4),
        CodEjemplar number(3),
       fecha_creacion DATE DEFAULT SYSDATE,
        Estado char(1) NOT NULL,
        CONSTRAINT pk_ejemplar PRIMARY KEY (CodLibro, CodEjemplar)
);
```

La guerra se complica…

Restricciones FOREIGN KEY I

Las restricciones FOREIGN KEY permite establecer las restricciones entre las tablas. Su sintaxis es:

CONSTRAINT [nombre_restriccion] **FOREIGN KEY**(columnas) **REFERENCES** tabla [(columnas)] [ON DELETE CASCADE|ON DELETE SET NULL]

Restricciones FOREIGN KEY II (Ejemplo)

```
CREATE TABLE Matricula(
        CodAsig number(4),
        CodAlumn number(4),
        CONSTRAINT pk_matricula PRIMARY KEY(CodAsig, CodAlumn),
        CONSTRAINT fk_matricula_asig FOREIGN KEY (CodAsig) REFERENCES Asignatura(CodAsig),
        CONSTRAINT fk_matricula_alumn FOREIGN KEY (CodAlumn) REFERENCES Alumno(CodAlumn)
);
```

```
CREATE TABLE Matricula(
        CodMatricula number(4) PRIMARY KEY,
        CodAsignatura number(4),
        CodAlumn number(4),
        CodMatriculaRecomendada number(4),
        CONSTRAINT fk_matricula_asig FOREIGN KEY (CodAsignatura) REFERENCES Asignatura(CodAsig),
        CONSTRAINT fk_matricula_alumn FOREIGN KEY (CodAlumn) REFERENCES Alumno(CodAlumn),
        CONSTRAINT fk_matricula_recomendada FOREIGN KEY (CodMatriculaRecomendada ) REFERENCES Matricula(CodMatricula)
);
```

Restricciones de integridad I

- Son optativas.
- Se crean para controlar el **borrado** o **puesta a nulo** de una FK.

Asignatura		
CodAsig	Nombre	NumHoras
1	BBDD	160
2	Prog	190

Matricula	
CodAlumn	CodAsig
1	1
1	2
2	2

Alumno		
CodAlum	Dni	NombreComplet
1	11111111K	Javier López Sánz
2	22222222L	María Domínguez Ramos

Restricciones de integridad II

- Para obligar a su eliminación en cascada, existen dos tipos:
  - **ON DELETE CASCADE**. Si se desea que todos los registros que apunten al registro eliminado también sean eliminados.
  - **ON DELETE SET NULL**. En lugar de eliminar un registro lo que hace es colocar el valor NULL en su lugar.
  - **ON DELETE SET DEFAULT**. En este caso perimitrá tomar el valor por defecto de la columna en BBDD **Oracle no permite esta opción**.

Restricciones de integridad III (Ejemplo)

```
CREATE TABLE Matricula(
        CodAsig varchar(4),
        CodAlumn number(4),
        CONSTRAINT pk_matricula PRIMARY KEY(CodAsig, CodAlumn),
        CONSTRAINT fk_matricula_asig FOREIGN KEY (CodAsig) REFERENCES Asignatura(CodAsig) ON DELETE CASCADE,
        CONSTRAINT fk_matricula_alumn FOREIGN KEY (CodAlumn) REFERENCES Alumno(CodAlumn) ON DELETE CASCADE
);
```

Restricciones de integridad IV

- La actualización de los valores de las claves ajenas también se puede determinar. Es decir si el CodAsig 1 cambia a ser 3, para que esto se realice en todos los registros se deberá añadir la sentencia:

  ON UPDATE CASCADE

  ¡Ojo! Esta cláusula y ON DELETE SET DEFAULT no están disponibles en ORACLE pero sí en otros SGBD.

Restricciones de integridad V (**Ejemplo NO válido para Oracle)**

```
CREATE TABLE Matricula(
        CodAsig number(4),
        CodAlumn number(4),
        CONSTRAINT pk_matricula PRIMARY KEY(CodAsig, CodAlumn),
        CONSTRAINT fk_matricula_asig FOREIGN KEY (CodAsig) REFERENCES Asignatura(CodAsig) ON DELETE CASCADE ON UPDATE CASCADE,
        CONSTRAINT fk_matricula_alumn FOREIGN KEY (CodAlumn) REFERENCES Alumno(CodAlumn) ON DELETE SET DEFAULT ON UPDATE CASCADE
);
```

Riesgos de usar ON UPDATE CASCADE

Restricciones de validación (CHECK) I

- Se usan para controlar el dominio de los valores de una columna.

CONSTRAINT ck_genero **CHECK** (Genero IN('Narrativo','Lírico', 'Dramático', 'Didáctico'))

```
CREATE TABLE Libro(
        CodLibro number(4),
        ISBN number(14) UNIQUE,
        Titulo varchar(32) NOT NULL,
        NumPags number(4),
        Genero varchar(32),
        CONSTRAINT pk_libro PRIMARY KEY(CodLibro),
        CONSTRAINT ck_genero CHECK (Genero IN ('Narrativo','Lírico', 'Dramático', 'Didáctico'))
);
```

Restricciones de validación (CHECK) II

- Otra forma de definir el CHECK es a la hora de crear la columna:

```
CREATE TABLE Libro(
        CodLibro number(4),
        ISBN number(14) UNIQUE,
        Titulo varchar(32) NOT NULL,
        NumPags number(4),
        Genero varchar(32) CONSTRAINT ck_genero CHECK(Genero IN('Narrativo','Lírico', 'Dramático', 'Didáctico')),
        CONSTRAINT pk_libro PRIMARY KEY(CodLibro)
);
```

Restricciones de validación (CHECK) III

- También se puede crear si nombrar la restricción, pero no se recomienda este método ya que a posteriori no podrá ser modificada.

```
CREATE TABLE Libro(
    CodLibro number(4),
    ISBN number(14) UNIQUE,
    Titulo varchar(32) NOT NULL,
    NumPags number(4),
    Genero varchar(32) CHECK(Genero IN('Narrativo','Lírico', 'Dramático', 'Didáctico')),
    CONSTRAINT pk_libro PRIMARY KEY(CodLibro)
);
```

¡NO Se REcomienda!

Restricciones de validación (CHECK, BETWEEN, IN, AND, OR) IV

- Otras restricciones comunes usada son:
  - CONSTRAINT ck_sup **CHECK** (superficie > 0 **AND** esAlquiler **IN** ('S','s','N','n'))
  - CONSTRAINT ck_prod descuento **BETWEEN** 0 **AND** 100 AND IVA **IN** (4,10,21)
  - Restringiendo un valor entre [1-250)
    CONSTRAINT ck_linea **CHECK** (cantidad > 0 **AND** cantidad < 250)
  - Restringiendo un valor entre [0,01 - 9999.99]
    CONSTRAINT ck_precio **CHECK** (precio **BETWEEN** 0.01 **AND** 9999.99)
  - Usando OR
    CONSTRAINT ck_genero CHECK(Genero IN('Narrativo','Lírico', 'Dramático', 'Didáctico') OR Genero IN('Comedia', 'Novela'))

Restricciones BOOLEAN I

Los campos booleanos no se pueden definir a la hora de crear las columnas de las tablas en ORACLE.

Para ello, se simula su comportamiento mediante char, number o varchar:

```
CREATE TABLE Matricula(
        CodAsig number(4),
        CodAlumn number(4),
        esPagada char(1) check (esPagada in ('Y','N')),
        CONSTRAINT pk_matricula PRIMARY KEY(CodAsig, CodAlumn),
        CONSTRAINT fk_matricula_asig FOREIGN KEY (CodAsig) REFERENCES Asignatura(CodAsig),
        CONSTRAINT fk_matricula_alumn FOREIGN KEY (CodAlumn) REFERENCES Alumno(CodAlumn)
);
```

Restricciones BOOLEAN II

```
CREATE TABLE Matricula(
        CodAsig number(4),
        CodAlumn number(4),
        esPagada number(1) check (esPagada in (1,0)),
        CONSTRAINT pk_matricula PRIMARY KEY(CodAsig, CodAlumn),
        CONSTRAINT fk_matricula_asig FOREIGN KEY (CodAsig) REFERENCES Asignatura(CodAsig),
        CONSTRAINT fk_matricula_alumn FOREIGN KEY (CodAlumn) REFERENCES Alumno(CodAlumn)
);
```

```
CREATE TABLE Matricula(
        CodAsig number(4),
        CodAlumn number(4),
        esPagada varchar2(5) check (esPagada in ('true','false')),
        CONSTRAINT pk_matricula PRIMARY KEY(CodAsig, CodAlumn),
        CONSTRAINT fk_matricula_asig FOREIGN KEY (CodAsig) REFERENCES Asignatura(CodAsig),
        CONSTRAINT fk_matricula_alumn FOREIGN KEY (CodAlumn) REFERENCES Alumno(CodAlumn)
);
```

+ Restricciones

```
CONSTRAINT ck_duracion CHECK (Duracion_Horas > 0),

CONSTRAINT ck_anio_academico CHECK (REGEXP_LIKE(Anio_Academico, '^[0-9]{4}/[0-9]{4}$')),

CONSTRAINT ck_fechas CHECK (Fecha_Fin > Fecha_Inicio),

CONSTRAINT ck_monto_compra CHECK (Monto_Total IS NULL OR Monto_Total >= 0),

CONSTRAINT ck_email_sede CHECK (Email IS NULL OR Email LIKE '%@%')

CONSTRAINT ck_marca_camara CHECK(LENGTH(Marca) > 0),
```

Modificación y eliminación de tablas

Observaciones generales

- Ya se ha analizado cómo se pueden crear tablas con CREATE TABLE
- Para modificar o eliminar se usará ALTER o DROP (CREALDROP)
- Estos comandos pertenecen al DDL.
- Estas instrucciones **no tienen vuelta atrás** o **ROLLBACK**
- Ya que contienen un **COMMIT** implícito.
- Antes de modificar la estructura de una BD es importante analizar **cómo están los datos** almacenados y siempre que sea posible su modificación no debería cambiar estos datos.
- En ocasiones son necesarios procesos intermedios de **exportación** e **importación**.

Modificar columnas y restricciones (ALTER TABLE)

- Para modificar una tabla creada.
- Para añadir, modificar o eliminar una columna.
- Para añadir, modificar o eliminar una restricción.

Se usa el comando **ALTER TABLE**

Su sintáxis es:

ALTER TABLE [esquema].tabla {ADD|MODIFY|DROP {definición_columna|CONSTRAINT definición_restrinccion}}

Modificación columnas ejemplos I

- Para añadir una columna a una tabla:
  ALTER TABLE Ediccion ADD numTiradas Number(5);

- Para añadir una restricción a una columna:
  ALTER TABLE Ediccion ADD CONSTRAINT ck_numTiradas CHECK (numtiradas >=0 AND numtiradas<=99999);

- Para modificar una columna:
  ALTER TABLE Ediccion MODIFY numTiradas Number(7) NOT NULL DEFAULT 0;

Modificación columnas ejemplos III

- Para modificar una restricción:
  ALTER TABLE Ediccion MODIFY CONSTRAINT ck_numTiradas CHECK (numtiradas >=0 AND numtiradas<=9999999);

- Para eliminar una restricción:
  ALTER TABLE Ediccion DROP CONSTRAINT ck_numTiradas;

- Para eliminar una columna de una tabla:
  ALTER TABLE Ediccion DROP NumTiradas;

Modificación columnas ejemplos IV

- Para cambiar el nombre de una **columna** o de una **restricción** se usa:
  ALTER TABLE Ediccion **RENAME** numTiradas **TO** numeroTiradas;

- Para cambiar el nombre de una tabla o cualquier objeto de la BBDD:
  RENAME Ediccion **TO** Periodico

- Para Eliminar unta tabla, se debe revisar:
  - ¿Tiene registros en su interior? Con este comando se eliminan todos los registros.
  - ¿Los registros que contienen están siendo referenciados? Si es así no se podrá eliminar almenos que se use la clausula CASCADE CONSTRAINTS.
  - Al eliminar una tabla, no se elimina su espacio ocupado, para ello que se libere lo antes posible se deberá añadir la opción PURGE.

  DROP TABLE Ediccion **CASCADE CONSTRAINTS PURGE**;

Modificación columnas ejemplos V

- Para **vaciar** de registros una tabla, se usa **TRUNCATE**:
  TRUNCATE Edicción

- Para activar o desactivar las restricciones de una tabla:

  ALTER TABLE Ediccion **DISABLE CONSTRAINT** ck_numtiradas CASCADE;
  ALTER TABLE Ediccion **ENABLE CONSTRAINT** ck_numtiradas CASCADE;

  En estos casos se añade CASCADE para desactivar o activar todas las restricciones que dependan de ellas.

¿Y qué se verá en la U4.2?

U4.2 Diseño Físico de BBDD (Parte 2)

Preparación de entorno

Usando Oracle Live SQL

¿Y qué se verá en la U4.3?

U4.3 Diseño Físico de BBDD (Parte 3)

Lenguaje de manipulación de datos (DML)

Select, Insert, Delete, Update…

Lenguaje de control de datos (DCL)

Grant, Revoke…

¡Ahora toca practicar!